

Shakespeare AI: Scholar Studio

Project Vision, Architecture, and Technical Specifications Report

The Story & Vision: In an era dominated by transient digital feeds and fleeting headlines, the endurance of classical literature stands as a testament to the depth of human self-reflection. Shakespeare AI was conceived to bridge the gap between four centuries of poetic genius and modern agentic computing. Crucially, the project is animated by the warning that **we will always be doomed to repeat history unless we study how its patterns are here in the present**. We are not only learning Shakespeare, but also finding analogous situations that were present even in the history of the bard—discovering echoes of our modern struggles in the very words and biography of the playwright. Co-developed alongside the **Antigravity** coding agent, this secure workspace integrates offline-first database systems, live search-grounded Gemini intelligence, and a drifting, retro-cybernetic (Max) Shakespeare (Headroom).

1. Agent & Multi-Agent System (ADK)

The core conversational logic of Shakespeare AI is built entirely on the **Agent Development Kit (ADK)** framework. Under this paradigm, the main entry point runs a decoupled FastAPI application that invokes the ADK **InMemoryRunner** to manage session states and history events. The agent logic itself, declared in `agent.py`, is structured as an ADK **Workflow** graph. The coordinator node (`intent_analyzer`) parses user messages into a structured Pydantic `RouteSelection` schema, routing requests dynamically to specialized subagents: 1) *Corpus Reader* (accesses the SQLite text database using the `query_play_dialogue` tool); 2) *Corpus Searcher* (finds occurrences via `search_shakespeare_text`); 3) *History Scholar* (answers biographical questions via `read_historical_context`); 4) *Scholarship Agent* (executes live searches via the `search_web_for_scholarship` tool); 5) *Advice Guide* (dealing with finding quotes and sayings that match the news or climate stories); and 6) *General Chat* (runs the Bard's friendly persona).

2. The Antigravity Developer Agent

The **Antigravity** developer agent acted as the primary engineer of creation for the prototype. Antigravity drove the entire development lifecycle: 1) *UI and Animation Engineering*: Resolved viewport scroll lock issues on mobile devices (e.g., Pixel 8) by editing `index.css` to support responsive overflow behaviors, and engineered the Max Headroom bouncing head animation in `index.js` with cache-busting version tags in `index.html`; 2) *Asset Optimization*: Generated and ran a flood-fill background-removal script (`remove_background.py`) to scrub checkerboard pixels from the user's Shakespeare portrait, creating a transparent `shakespeare_head.png`; 3) *Security & Environment*: Configured production GCP Secret Manager settings and programmatically updated Cloud Run deployments.

3. Model Context Protocol (MCP) Server Integrations

During the development phase, the agent relied heavily on the **Model Context Protocol (MCP)** to query the local developer ecosystem. Specifically, the `google-developer-knowledge` MCP server was accessed to: 1) *Fetch GCP Best Practices*: Retrieved detailed references for Secret Manager IAM policy bindings and environment variable configurations; 2) *Cloud Run Security Specs*: Consulted reference guides to configure unauthenticated public invokers on GCP safely; 3) *FastAPI/ADK Syntax*: Resolved integration questions between FastAPI endpoints and the ADK runner async generators, ensuring clean session teardowns.

4. Security Features & Session Isolation

Security is integrated at multiple levels of the codebase: 1) *Cryptographic Hashing*: User credentials are encrypted at registration using the PBKDF2 algorithm with native `hashlib.pbkdf2_hmac` and a random 16-byte salt, stored in an SQLite user database; 2) *Session Isolation*: To eliminate session bleeding in a stateless container, the client generates a unique UUID on load passed in a custom X-Session-ID header. The FastAPI backend intercepts this header, using it to route database operations to the correct session scope and log telemetry events (`ip_address`, `location`, `query`, `event_type`) in isolation; 3) *Semantic Guardrails*: Input text is checked via `check_input_safety` before model calls, leveraging a safety-specialized Gemini configuration to block prompt injection (e.g., 'ignore instructions') with regex fallbacks.

5. Agent Skills & Test Automation (Agents CLI)

The development, validation, and deployment phases leverage advanced **Agent Skills**: 1) *Agents CLI*: Project configuration is scaffolded using `agents-cli-manifest.yaml`. Local syntax checks and quality audits are executed via `agents-cli lint`; 2) *Unit Testing*: Automated testing is done via `test_backend.py` with `pytest`, executing credentials verification and guardrail checks; 3) *Cloud Deployments*: The Docker compilation and deployment to Cloud Run are managed using automated deployment scripts, allowing transition from local host to public production.

6. Overall User Value

For scholars, educators, and students, Shakespeare AI delivers high-fidelity literary research in a secure environment. It provides instant text exploration, AI-assisted comparative essay writing, and grounded current events critique. By recognizing that we are doomed to repeat history unless we study its patterns in the present, it offers a workspace that bridges literature, history, and modern agentic safety into a unified, mobile-optimized experience.

Shakespeare AI: Technical Assessment Report

Architecture Quality, Code Integrity, and Agent/Tool Evaluations

Executive Assessment: This document provides an independent technical evaluation of the Shakespeare AI workspace. The prototype exhibits a robust, production-ready decoupled architecture built around the Google Agent Development Kit (ADK). The code is characterized by strict session isolation, semantic safety guardrails, elegant tool design (including dual-step search grounding), and a fully commented codebase. This evaluation grades the architectural implementation quality as high-fidelity and secure.

1. Architectural Quality & Session Isolation

The codebase demonstrates a decoupled design that separates the client interface (HTML5/CSS3/JS) from the backend server (FastAPI). State management is securely managed using an SQLite database file (shakespeare.db). Rather than relying on cookies or server sessions which are prone to leaks and session bleeding in stateless containerized services like Google Cloud Run, the system uses a custom **X-Session-ID** header. The client generates a unique UUID on page load and includes it with every HTTP request. The FastAPI backend intercepts this header to partition the database telemetry logs, creating absolute session isolation and logging detailed events (IP, location, debounced scroll markers, and queries) in isolation.

2. Multi-Agent Design & ADK Graph

The conversational engine utilizes the **Agent Development Kit (ADK)** to construct a structured `Workflow` graph. This design avoids monolithic agent loops. Instead, it delegates user intents to specialized subagents. The graph's entry point is the `intent_analyzer` node, which parses inputs against a strict Pydantic schema (`RouteSelection`), directing queries to Corpus Reader, Corpus Searcher, History Scholar, Scholarship Agent, Advice Agent, or General Chat. This structured routing model minimizes prompt ambiguity and provides highly predictable conversational flows, making the multi-agent system both stable and testable.

3. Clever Tool Usage (Dual-Step Search Grounding)

A major technical highlight of the implementation is the clever use of Gemini's search grounding tool within the current events and scholarship nodes. Traditional search tools feed raw web search results directly to the model, which often violates UI JSON output constraints. This project implements a **Dual-Step Grounding Pattern**: 1) *Grounding Step*: The model is called with the Google Search tool enabled to fetch relevant, citation-rich context from the web. 2) *Parsing Step*: The output of the first step is fed into a separate structured generator call with tools disabled, enforcing a strict JSON schema contract (`TopNewsResponse` or `NewsStory`). This pattern isolates web discovery from schema parsing, yielding highly reliable structured outputs without violating data contracts.

4. Clever Caching & Token Management

To prevent rapid API token drainage and avoid redundant external requests during server hot-restarts or rapid client refreshes, the system implements an intelligent file-age cache check in the background `news_cache_refresher_loop`. Before making live searches, the app inspects the file modification time of the local caches (news_cache.json / climate_cache.json). If the cache file is less than 12 hours old (configured during Vertex deployment to conserve API quota, returning to 15 minutes when live), the backend serves the cached data immediately. This clever utility reduces external model calls, preserving token bandwidth while maintaining fresh current events feeds.

5. UI Scroll Optimizations & Assets

Frontend assets are structured with responsive styling rules in `index.css`. Viewport layouts for mobile touch screens (such as the Pixel 8) are optimized using CSS flex overflow limits to prevent viewport lockouts and scrolling freezes. Furthermore, the floating Shakespeare asset (`shakespeare_head.png`) was optimized using a custom BFS-based flood-fill script to strip checkerboard canvas grids and establish clean alpha transparency, enabling a smooth Max Headroom-style bouncing drift animation in JavaScript.

6. Code Quality, Comments & Clean Design

The code is thoroughly documented with comments detailing implementation details, design rationales, and runtime behaviors. In `main.py` and `agent.py`, comments clearly explain: 1) why PBKDF2 hashing is chosen for auth; 2) how custom headers separate user telemetry; 3) how pre-flight checks protect LLM nodes from prompt hijacking; and 4) how the ADK workflow handles session teardowns and event logs. This comprehensive commenting ensures long-term maintainability and readability for new developers importing the codebase.